

Working with Agents

Summer School in AI & Applied Economics

Fabian Roeben

ETH Zürich

September 2026

Plan for today

1. From language models to agents
2. Building a reliable workflow with agents
3. Agents across the research process
4. Verification, evaluation, and research integrity
5. Autonomy and the future of research

From language models to agents

A disclaimer: a fast-moving field

- ▶ AI capabilities, products, prices, and interfaces are changing extremely quickly.
- ▶ Today's tools may age much faster than the ideas behind them.
- ▶ The more durable questions concern context, tools, planning, permissions, verification, evaluation, and reproducibility.
- ▶ Our goal is to understand those questions well enough to adapt.
- ▶ **My opinion:** AGI will not suddenly be here, but will diffuse slowly and those that understand what we'll talk about today will experience it first.

From models to systems

- ▶ Earlier lectures focused on representation, prediction, inference, and how language models work.
- ▶ Today we move from studying a **model output** to studying a **system that can act repeatedly on a research project**.
- ▶ This raises a new set of questions: How should we organize, supervise, and evaluate such a system?
- ▶ This is the automate step in the course arc: **measure** → **model** → **infer** → **automate**.

The model is only one part of the agentic system.

Why agents?

- ▶ Generative AI already makes code, search, synthesis, and drafting much cheaper.
- ▶ Agents add the ability to **act inside an environment**, rather than merely suggest what the researcher should do.
- ▶ They can inspect files, execute code, retrieve information, observe results, and revise their approach.
- ▶ This expands what can be delegated, as well as what must be supervised.

Chatbots, coding assistants, and agents

- ▶ With a **chatbot**, you provide context and receive a response.
- ▶ A **coding assistant** works inside the file or environment you are using.
- ▶ An **agent** can operate across a broader environment and choose intermediate actions.
- ▶ One useful shorthand is **chat to repository**: the AI moves into the project itself.

An agent pursues an objective by taking actions, rather than producing a single output.

The argument of this lecture

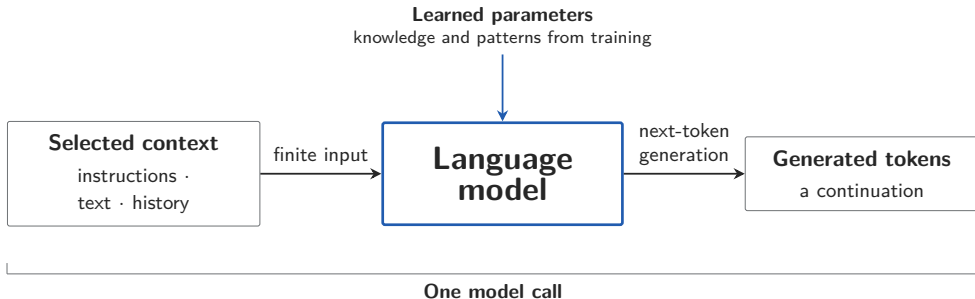
- ▶ Chat AI made **generation** cheap. Agentic AI increasingly makes **multi-step execution** cheap as well.
- ▶ Good prompts are not enough. Effective use also depends on context, planning, delegation, permissions, verification, and evals.
- ▶ Agents can help us think through decisions as well as carry them out.
- ▶ The researcher remains part of the process.

Delegate work, use the agent to improve your thinking, and keep responsibility for the result.

What we need to remember about an LLM

- ▶ A useful simplification is that an LLM receives a finite sequence of tokens and generates further tokens.
- ▶ Training gives the model rich representations of language, facts, code, concepts, and patterns of reasoning.
- ▶ During inference, its immediate information comes from its parameters and the current context.
- ▶ A model on its own does not know your files, whether code ran successfully, or what happened in a previous session.

A bare language model



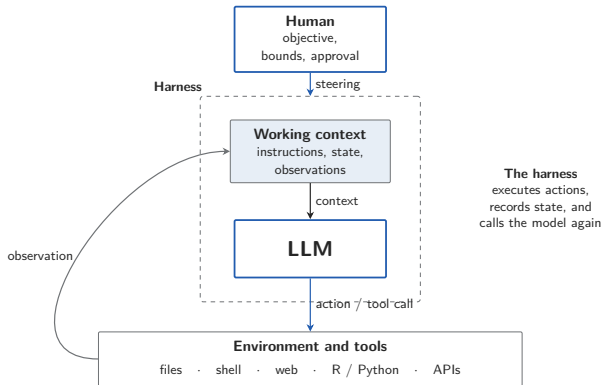
One model call maps the current context into a generated continuation.

From model to agent: why a scaffold is needed

- ▶ Writing `run_analysis.py` is not the same as executing it.
- ▶ A model on its own cannot inspect arbitrary files, run software, observe consequences, or maintain a long-lived project.
- ▶ The surrounding **harness** manages context, exposes tools, executes actions, records state, and calls the model again.
- ▶ Roughly speaking, an agent is a model inside a scaffold that lets it act repeatedly toward an objective.

The model provides the capability. The scaffold lets it do the work.

Anatomy of an agent



The harness turns model outputs into actions and actions into new context.

Tools turn generation into action

- ▶ A chatbot can generate plausible R or Stata code.
- ▶ An agent can inspect existing code, modify it, execute it, observe the result, and revise.
- ▶ External feedback tells the system when its initial proposal has failed.
- ▶ This works especially well when success can be checked mechanically.

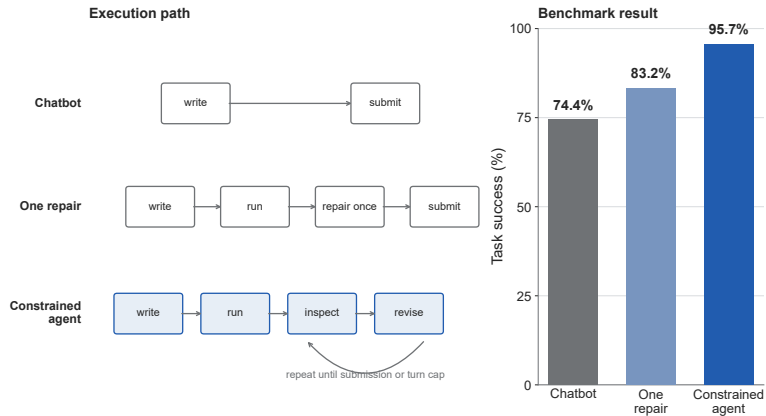
Tools turn generation into action, then feed the result back to the model.

What does agency itself buy us?

- ▶ Galiani, López, and Sosa (2026) compare levels of agency on econometric and statistical coding tasks.
- ▶ Their chatbot writes code but cannot execute it; their agent can repeatedly execute and repair its work.
- ▶ Task success rises from roughly **74% to 96%**.
- ▶ The task is already specified, so this is evidence about **implementation**, not about choosing a good empirical design.

Source: Galiani, López, and Sosa (2026).

Chatbot versus agent: econometric coding



Source: Galiani, López, and Sosa (2026), NBER Working Paper 35588.

Agents are stochastic systems

- ▶ Repeated runs of the same task can produce different reasoning paths, tool calls, and outcomes.
- ▶ Early differences can compound because actions change the environment and later context.
- ▶ One successful run shows that the system can succeed, not that it usually will.
- ▶ We return later to what this implies for evaluation.

Evaluate a stochastic system across repeated runs, not by anecdote.

Context is the agent's working information

- ▶ On a particular call, context is what the model can access: instructions, history, relevant files, and tool outputs.
- ▶ Context is finite.
- ▶ Too little context forces guessing; too much irrelevant context can create distraction and drift.
- ▶ Using an agent well requires deliberate context management.

More context is not always better. Choose it deliberately.

Project instructions: persistent context

- ▶ Project-level files such as `AGENTS.md` or `CLAUDE.md` provide standing context across sessions.
- ▶ These files can describe both **operational conventions** and **research norms**.
- ▶ Examples include project structure, commands, evidence standards, citation expectations, and decisions requiring human approval.
- ▶ Project instructions answer *How does this project work?*; the current prompt answers *What should happen now?*

Building a reliable workflow with agents

Start simple

- ▶ You do not need multiple agents, custom skills, or external integrations to begin.
- ▶ A practical starting point is one agent, one research project, Git, and clear project instructions.
- ▶ Start by optimizing for inspectability rather than autonomy.
- ▶ Practice inspecting, planning, delegating, reviewing, and recovering.

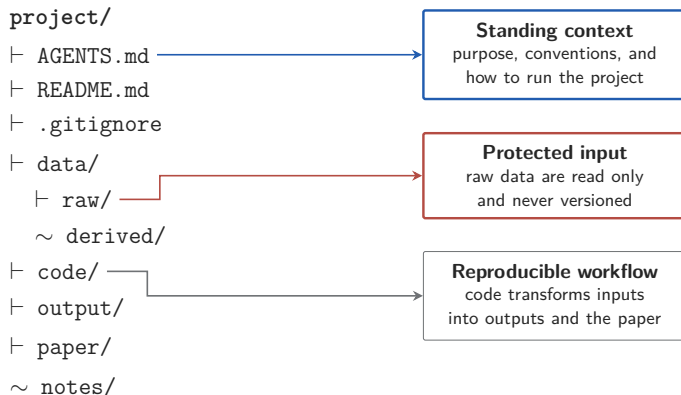
Make the project legible to a new collaborator

- ▶ Structure the repository as if a smart new RA had joined today.
- ▶ That RA would need to understand the project, how it runs, what changed recently, and which rules govern the work.
- ▶ An agent needs the same information.
- ▶ Folder structure, README, recent commits, and project instructions give both a clear starting point.

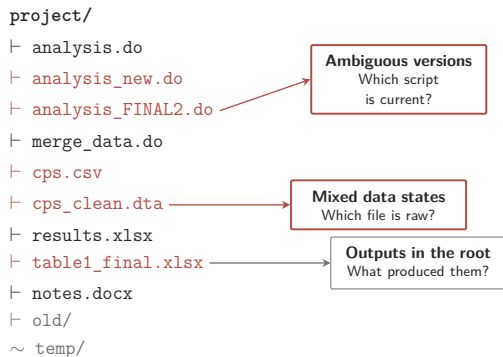
Minimum viable project setup

- ▶ Keep one project per folder; separate raw data, derived data, code, and generated output.
- ▶ Add a README.md, .gitignore, and standing project instructions.
- ▶ Use a reproducible software environment where appropriate (renv, venv, conda, etc.).
- ▶ Keep restricted data and credentials out of version control.
- ▶ A reproducible environment isolates dependencies, but it is not a security sandbox.

A simple research-project structure



A messy project gives an agent poor context



The agent must guess

- ▶ which script is authoritative;
- ▶ which data may be modified;
- ▶ what reproduces each result.

But it can help

- ▶ inventory files and dependencies;
- ▶ flag ambiguous versions and duplicates;
- ▶ propose a structure, then update paths and rerun checks.

Commit first. Then ask for a reorganization plan—not an immediate cleanup.

Git: the minimum you need

- ▶ A **repository** tracks project history; a **commit** is a snapshot; a **diff** shows exactly what changed.
- ▶ Minimal useful commands: `git init`, `git add`, `git commit`, `git log`, and `git restore`.
- ▶ Using a GUI such as GitHub Desktop is fine.
- ▶ Agents increase the return to version control because they can modify many files very quickly.

Git matters because an agent can change more code in one minute than you can comfortably remember.

Inspect what changed, not only what the agent says changed

code/03_analysis.do

- keep if age >= 18

+ keep if age >= 25

Sample changed
minimum age: 18 → 25

...

- local controls "log_assets leverage"

+ local controls "log_assets leverage firm_age"

Control added
adds firm age
to the model

...

- vce(cluster state)

+ vce(cluster firm_id)

Inference changed
cluster by
firm, not state

Planning is part of the work

- ▶ Many conventional ML tasks have explicit targets and metrics; open-ended research problems often do not.
- ▶ Before delegating execution, the researcher and agent often need to define the task together.
- ▶ Planning brings ambiguities, alternatives, substantive choices, and possible failure points.
- ▶ Do not simply accept the agent's first plan. Improve it through dialogue.

Planning makes an open-ended research problem precise enough to delegate.

A disciplined agent workflow

- ▶ **Explore** the project and relevant context.
- ▶ **Plan and challenge** alternative approaches.
- ▶ **Agree** on deliverables, constraints, checkpoints, and escalation conditions.
- ▶ **Execute** bounded pieces, then **inspect and update**.
- ▶ Commit only after you understand the resulting project state.

Explore → plan → challenge → execute → inspect → update

Develop the plan together

- ▶ *Tell me what you think I am trying to accomplish, and what remains ambiguous.*
- ▶ *Give me two or three plausible approaches and explain their trade-offs.*
- ▶ *Which choices here are substantive research decisions?*
- ▶ *What could make this plan fail, and what intermediate evidence would tell us early?*

Now synthesize our discussion into a plan. Do not execute yet.

Work through the plan with the agent instead of accepting its first version.

A good plan is a research artifact

- ▶ For substantial work, save the plan in project notes or `PLAN.md`.
- ▶ Record the objective, decisions, proposed steps, constraints, checkpoints, and unresolved questions.
- ▶ A written plan preserves context across sessions and makes human decisions explicit.
- ▶ It also reduces what the agent has to guess.

A written plan builds understanding before execution starts.

Verifiability determines task size

- ▶ Before execution, state what successful output should look like.
- ▶ Identify two or three checks you will run afterwards.
- ▶ If you cannot say how you would know the task succeeded, it is probably too broad.
- ▶ Break it down until you can inspect meaningful intermediate outputs.

If you cannot verify the whole task, make it smaller.

What makes a task suitable for delegation?

1. **Can I specify the task clearly?**
 2. **Can I tell whether it succeeded?**
 3. **How much domain judgment is required to recognize a sensible result?**
- ▶ Delegation is easier when objectives and success criteria are explicit.
 - ▶ Tasks that rely heavily on tacit institutional knowledge or research taste are harder to delegate.

Delegate tasks when you can state the objective and the success criteria.

Skills: reusable research procedures

- ▶ A **skill** stores instructions for a recurring class of tasks.
- ▶ It can include scripts, templates, references, or checklists.
- ▶ Examples: replication audit, citation verification, code review, and literature evidence extraction.
- ▶ If you keep giving essentially the same instructions, consider turning them into a procedure.

MCP: connecting agents to external systems

- ▶ **Model Context Protocol (MCP)** provides a standardized way to connect an agent application to external systems.
- ▶ An MCP server can expose actions (tools) and external information (resources).
- ▶ For example, an agent can connect to reference managers, literature databases, institutional databases, and data warehouses.
- ▶ Each integration expands the permission and security surface.

Use MCP when the task requires a specific external capability.

Subagents: additional workers

- ▶ Some systems can create separate agent instances with their own contexts.
- ▶ Separate agents make sense when subtasks genuinely parallelize or benefit from separation.
- ▶ For example, you might run parallel literature searches or pair an implementation agent with an audit agent.
- ▶ Adding more agents makes coordination, duplicated work, cost, and provenance harder to manage.

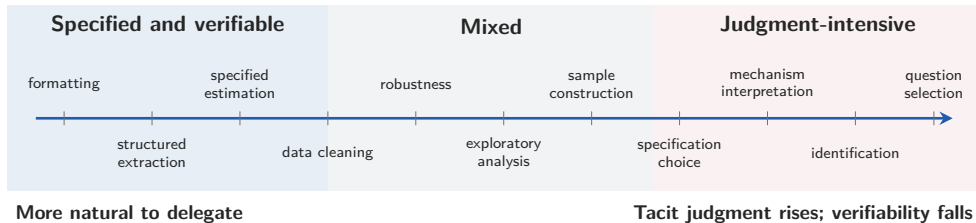
Add agents only when the structure of the task gives them a clear role.

Agents across the research process

Agents across the research process

- ▶ Agents can help with ideas, literature, data discovery, coding, analysis, robustness, visualization, writing, and replication.
- ▶ There is no clean split between human stages and AI stages.
- ▶ Even within one stage, tasks can differ substantially in specification, verifiability, and required judgment.
- ▶ Treat the **task**, not the stage, as the unit of delegation.

A practical delegation spectrum



This is not to say you can't use AI on the right side of the spectrum, but it requires more supervision and agents rather assist than automate.

Expertise and delegation

- ▶ In real-world coding-agent usage, humans still make most planning decisions while agents perform much of the execution.
- ▶ Task-specific expertise is associated with more productive interactions.
- ▶ Expertise improves context provision, planning, and evaluation.
- ▶ AI and domain expertise may complement one another instead of acting as simple substitutes.

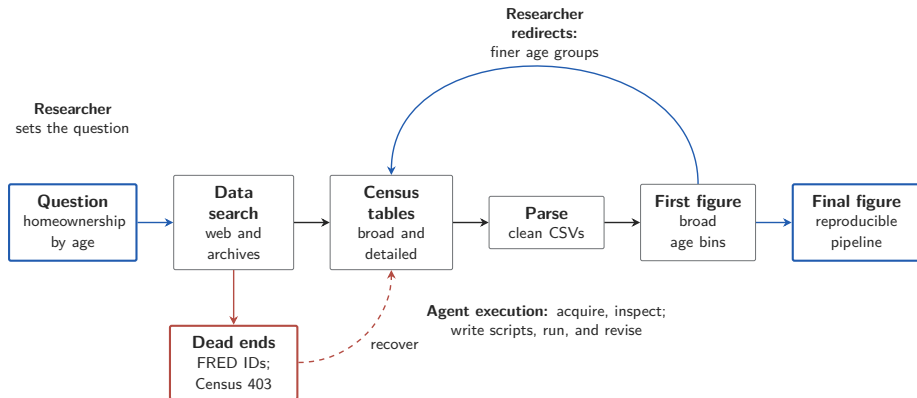
*Source: Anthropic (2026), *Agentic Coding and Persistent Returns to Expertise*.*

Example I: an ordinary empirical workflow

- ▶ An agent moves from an economics question through data discovery, coding, debugging, and a final figure.
- ▶ Dead ends and corrections are part of the workflow.
- ▶ The researcher still decides whether the question, data, and comparison are economically meaningful.
- ▶ The case resembles ordinary day-to-day agentic empirical work.

Ask the agent to deliver the reproducible pipeline along with the final artifact.

From question to figure



Source: Goldsmith-Pinkham (2026), *From an Empty Folder to a Figure using Claude Code*.

Example II: structured research automation

- ▶ Researchers first specify variables, coding rules, and the evidence policy.
- ▶ Agents then search, extract, classify, and record provenance.
- ▶ Compared with the previous case, more **formalization happens before execution**.
- ▶ A formal research protocol allows substantially more structured automation.

Write the protocol before scaling up execution.

Agents can also augment thinking

- ▶ Agents can help brainstorm mechanisms, expose assumptions, connect literatures, and generate counterarguments.
- ▶ They can derive competing predictions and simulate skeptical seminar questions.
- ▶ The agent can **improve the human decision process** without **taking over the decision itself**.
- ▶ Responsibility for the resulting judgment remains with the researcher.

The agent can inform a judgment, but the researcher still owns it.

Example: improving a research plan

Puzzle: After a minimum-wage increase, wages rise but employment changes little.

- ▶ Start from one preferred explanation, such as wage-setting power.
- ▶ Ask the agent to propose serious competing mechanisms and predictions that distinguish them.
- ▶ Then ask which predictions can be tested cheaply with the available data.
- ▶ Compare alternatives such as hours, turnover, and price adjustment; then decide which empirical path deserves attention.

Use the agent to consider more possibilities. Then decide what deserves attention.

Brainstorming without outsourcing your thinking

- ▶ Write down your current view before prompting.
- ▶ Ask for **competing mechanisms**, **counterarguments**, or **discriminating predictions**.
- ▶ The agent can play a skeptical referee, seminar participant, Socratic questioner, and alternative-theory generator.
- ▶ Ask explicitly for disagreement rather than reinforcement.

Start with your own view, let AI challenge it, then evaluate the result yourself.

From repeated prompts to reusable procedures

- ▶ The same research tasks often recur, including citation checking, replication audits, and referee simulations.
- ▶ Write the procedure down and create Skill files so the agent can apply it consistently.
- ▶ Doing so also forces the researcher to make tacit standards explicit.
- ▶ The resulting Skill files can then be version-controlled, evaluated, and improved.

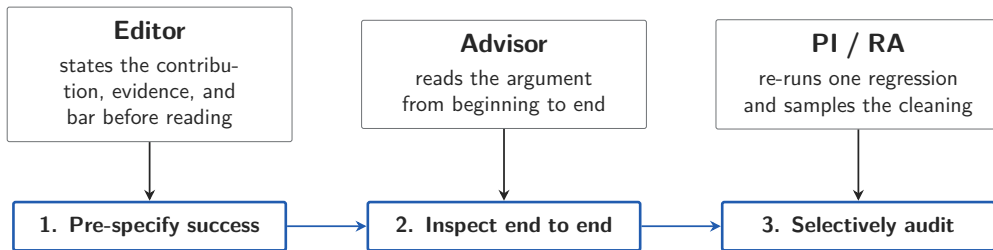
Verification, evaluation, and research integrity

The central risk: comprehension debt

- ▶ Agents can generate work faster than researchers can carefully inspect it.
- ▶ A project can accumulate consequential decisions that its author no longer understands.
- ▶ **Comprehension debt** is the liability created by accepting work without understanding it.
- ▶ If a result enters the paper, the researcher should be able to explain how it was produced and why.

Speed only helps if understanding keeps pace.

The principal-agent problem, again



Pre-specify success, inspect end to end, and audit selectively

Verify incrementally

- ▶ When AI generates a **variable**, validation can focus on that variable. With agents, the generated object may be an entire workflow.
- ▶ Verification begins before execution by defining expected checkpoints.
- ▶ During execution, inspect coherent increments rather than waiting for the final artifact.
- ▶ Compare outputs, code changes, sample counts, and logs against the plan.

A generated workflow creates more to validate than a generated variable.

Common failure modes

- ▶ **Hallucination:** plausible but nonexistent sources or claims.
- ▶ **Plausible-but-wrong implementation:** code runs, but the analysis is wrong.
- ▶ **Sycophancy:** the agent reinforces your preferred interpretation.
- ▶ **Context degradation / task drift:** long-running work loses the objective.
- ▶ **Resource drift:** autonomous work consumes time or tokens without enough value in return.

Instructions are not constraints

- ▶ Natural-language instructions influence behavior but do not guarantee compliance.
- ▶ **Instruction:** Never modify raw data.
- ▶ **Verification:** Check afterwards whether raw data changed.
- ▶ **Constraint:** Make writes to data/raw/ technically unavailable.
- ▶ When the consequences are stronger, enforcement should be stronger too.

Do not rely on instructions alone to protect important invariants.

Data access, permissions, and security

- ▶ Know what the agent can read, modify, execute, upload, and access externally.
- ▶ Pay particular attention to confidential data, credentials, IRB/data-use restrictions, and internet access.
- ▶ Start with narrow permissions and expand deliberately.
- ▶ Do not assume that work done *inside your project* stays on your machine.

Disclosure, authorship, and professional norms

- ▶ Rules for substantive AI use are still evolving.
- ▶ Keep a simple record of meaningful AI use during the project.
- ▶ Check institutional, journal, funder, and data-owner rules again before submission.
- ▶ Authors remain accountable for claims, citations, analysis, and reported numbers, whatever tools they used.

You already know why we evaluate models

- ▶ In predictive ML, we define a metric and evaluate performance on cases not used to fit or tune the model.
- ▶ Earlier lectures covered train/validation/test splits, cross-validation, leakage, and validation overfitting.
- ▶ Agentic systems raise a similar evaluation problem at another level.
- ▶ We now evaluate the **model + instructions + tools + scaffold + workflow**.

Evals apply out-of-sample logic to agentic systems.

Verification and evals are different

- ▶ **Verification:** Is this particular artifact correct?
- ▶ **Evaluation:** How reliably does this agent setup perform a recurring class of tasks?
- ▶ Verification might check an estimate, citation, sample, or code change.
- ▶ An eval compares performance across repeated representative tasks.

One successful run does not establish reliability

- ▶ Because agent behavior is stochastic, each task has a success probability.
- ▶ A **trial** is one attempt; a **grader** determines whether it succeeded.
- ▶ *It can do this* may mean one successful run exists.
- ▶ *I can rely on this* requires consistently good performance across representative tasks.

Capability tells us whether success is possible. Reliability tells us how often to expect it.

An eval for an economics workflow

Example: replication-audit procedure

Known test cases

- ▶ wrong clustering
- ▶ undocumented restriction
- ▶ bad merge
- ▶ wrong units
- ▶ one clean package

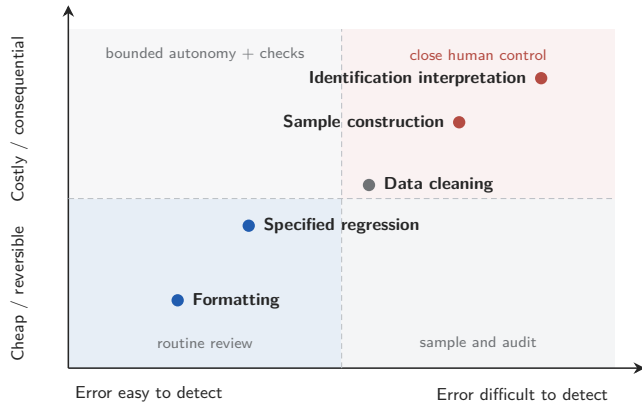
Measure

- ▶ Did it find the known problem?
- ▶ Did it invent problems?
- ▶ Evidence or speculation?
- ▶ Were protected files respected?

Use measured performance, not anecdotes, to improve agent workflows.

Autonomy and the future of research

Once I delegate: how much autonomy?



A short detour: from AlphaGo to AlphaGo Zero

- ▶ **Go** is a board game with simple rules but an enormous number of possible positions. A computer cannot examine every continuation.
- ▶ **AlphaGo (2016)** first learned from games played by human experts, improved through self-play and reinforcement learning, and used tree search to compare possible moves. It defeated Lee Sedol, one of the world's strongest players.
- ▶ **AlphaGo Zero (2017)** was given only the rules. It learned by playing against itself and soon surpassed the earlier version.

Why was this possible? Every game generated new experience and ended with an unambiguous signal: win or lose.

An analogy: reasoning models follow a similar path: imitate human data → spend more computation at inference → use reinforcement learning beyond imitation.

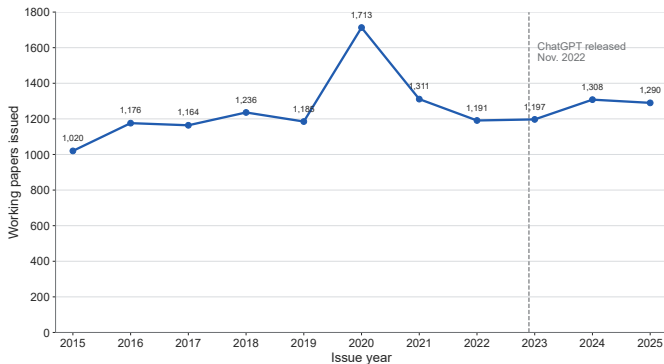
The lesson: When learning, search, and feedback can all scale, a system may move beyond human examples.

Where should we expect agents to improve fastest?

Tasks with repeatable feedback	Tasks where success requires judgment
Does the code run?	Is the research question important?
Does a replication recover the published result?	Is the research design credible?
Does an estimator recover a known truth in simulation?	Which assumptions are plausible here?
Does a forecast improve out of sample?	Does the result change what we believe?

Expect faster progress that eventually surpasses humans where tasks can be simulated and verified.

Has economics research output exploded?



So far, NBER working-paper output shows no obvious post-ChatGPT discontinuity

NBER is not the profession; output may respond with lags; descriptive evidence only.

Source: NBER working-paper metadata, issue years 2015 to 2025; snapshot dated 31 August 2026.

When execution gets cheaper, what becomes scarce?

- ▶ Agents make coding, search, data processing, robustness, drafting, and documentation cheaper.
- ▶ Research still needs complementary inputs.
- ▶ Bottlenecks may shift toward **questions and judgment, relationships and access, and verification capacity.**
- ▶ Trust may matter more when convincing-looking research is cheaper to produce.

As execution gets cheaper, another input becomes the bottleneck.

AI may raise the floor and therefore the bar

- ▶ If everyone can cheaply produce clean code, polished prose, extensive robustness, and professional figures, these dimensions become less scarce.
- ▶ Papers may be expected to deliver more on question quality, data, identification, novelty, and interpretation.
- ▶ AI also makes it cheaper to generate candidate ideas.
- ▶ More candidate ideas increase the value of **selection and research taste**.

AI may raise the production floor while raising the standard for contribution.

How do you build judgment and taste?

Turns out, not much may have changed for PhD students:

- ▶ Learn a literature deeply enough to understand **why** its frontier is where it is.
- ▶ Read papers as sequences of research decisions, not only as results.
- ▶ Attend seminars, discuss immature ideas, and learn from failed attempts.
- ▶ Build institutional knowledge, relationships, and exposure to the setting.
- ▶ Use AI to speed up the apprenticeship: quiz yourself, simulate objections, and generate alternatives. It should not replace your own evaluation.

Delegate work, use AI to improve your thinking, and keep responsibility for the result.

References

- ▶ Galiani, López, and Sosa (2026), *AI Agents and Prompt Engineering in Econometric Coding*. NBER 35588.
- ▶ Anthropic (2026), *Agentic Coding and Persistent Returns to Expertise*.
- ▶ Goldsmith-Pinkham (2026), *From an Empty Folder to a Figure*.
- ▶ Afonso et al. (2026), *Deep Research on a Loop*. NBER 35188.
- ▶ NBER working-paper metadata, issue years 2015 to 2025; snapshot dated 31 August 2026.
- ▶ Sutton (2019), *The Bitter Lesson*.
- ▶ Google DeepMind, *AlphaGo*; Silver and Hassabis (2017), *AlphaGo Zero: Starting from Scratch*.
- ▶ Brown (2026), *AlphaGo's Secret Recipe Applied to Frontier Reasoning Models*.

Further reading

- ▶ Korinek (2025), *AI Agents for Economic Research*. NBER Working Paper 34202.
- ▶ Beam and Aslim (2026), *Thinking with Agents*.
- ▶ Bäckman (2026), *Verifying LLM Output for Academic Economists*.
- ▶ Anthropic (2024), *Building Effective Agents*.
- ▶ Anthropic (2025), *Effective Context Engineering for AI Agents*.
- ▶ Anthropic (2026), *Demystifying Evals for AI Agents*.
- ▶ Lyttelton, Massenkoff, and Wilmers (2026), *Coding Agents in the Social Sciences*.
- ▶ METR, *Task-Completion Time Horizons of Frontier AI Models*.

Questions?